

Building an Air-Gapped Local “Codex-Type” AI Capability for Coding and Network Engineering Labs

Revised design excluding Chinese-origin and Chinese-distilled models; updated with Google Gemma options
Prepared for a disconnected Linux lab that supports medium-size software repositories, Cisco configuration work, Aruba SD-WAN appliance workflows, and command-line operation.

Executive Summary

This paper proposes a local, air-gapped AI coding and network-engineering assistant that can be built in an internet-connected staging environment and then transferred into a non-internet-connected lab. The design intentionally excludes Qwen, DeepSeek, Chinese-origin models, and Chinese-distilled derivatives. It uses non-Chinese open-weight candidates such as Meta Llama, Mistral Codestral, IBM Granite Code, and BigCode StarCoder2. The recommended runtime architecture separates model serving from the coding agent: vLLM or Ollama serves the model locally, while Aider, OpenCode, or a similar terminal coding agent performs repository-aware coding tasks.

The Cisco UCS C240 M5 can be used, but with important limitations. It is a PCIe Gen3, 2U platform that supports up to two double-wide GPUs in specific riser positions, and its practical modern-AI use is strongest with older supported GPUs such as NVIDIA T4/L4-class single-width cards, or carefully validated double-width cards such as legacy Tesla V100/A100 and Quadro RTX 6000/8000 configurations. For a stronger design that supports modern double-width AI GPUs such as NVIDIA L40S, H100 PCIe/NVL, or RTX PRO 6000-class hardware, this paper recommends considering Cisco UCS C240 M7, Dell PowerEdge R760xa, HPE ProLiant DL380a Gen11, Lenovo ThinkSystem SR675 V3, or Supermicro SYS-221GE-NR.

1. Design Goals and Non-Goals

Goal	Design Requirement
Air-gapped operation	After transfer, the system must not require internet, DNS, public package repositories, cloud APIs, or SaaS model endpoints.
Linux command-line workflow	The primary user interface should be SSH/terminal friendly, with optional web UI only inside the lab.
Medium-size codebase support	The assistant should index Git repositories, produce diffs, explain code, run tests, and work with multi-file changes.
Cisco and Aruba support	The assistant should generate change plans, parse configurations/logs, and use local vendor and internal documentation.
No Chinese-origin models	Exclude Qwen, DeepSeek, InternLM, Yi, Baichuan, MiniCPM, GLM, and any fine-tunes/distillations derived from those families.
Human approval for network changes	The AI can generate plans and configs, but humans approve before commands are applied to devices.

2. High-Level Architecture

```
Internet-connected staging area
- Clone repositories and pin versions
- Download model weights and tokenizer files
- Pull container images
- Build Python wheelhouse and npm cache
- Download Cisco/Aruba documentation
- Generate SBOM and SHA256 manifest
  |
  | signed/encrypted removable media
  v
```

Air-gapped lab

- Linux AI server/workstation
- Local model runtime: vLLM/Ollama/llama.cpp
- Terminal coding agent: Aider/OpenCode
- Local RAG: Cisco docs, Aruba docs, internal standards, golden configs
- Git workspaces and test harnesses
- No internet default route; firewall-deny egress

The architecture deliberately separates the local model server from the coding agent. vLLM is attractive because it provides an OpenAI-compatible HTTP server, which lets many tools talk to local models through a familiar API shape. Aider is a terminal pair-programming tool that can connect to local models and build a map of an entire codebase. OpenCode is provider-agnostic and can be pointed at local model endpoints. [1][2][3]

3. Model Policy: Non-Chinese Candidate Models Only

The following model shortlist avoids Chinese-origin and Chinese-distilled models. Final selection should still include a license review, hash pinning, and internal approval.

Model family	Origin / steward	Why it fits	Hardware fit
Mistral Codestral 22B	Mistral AI, France	Open-weight code model designed for code generation and interaction across 80+ programming languages. Strong candidate for coding CLI use.	Runs well on 48GB GPU; can run quantized on 24GB/32GB with reduced context.
IBM Granite Code 3B/8B/20B/34B	IBM, United States	Enterprise-oriented code models released under Apache 2.0 according to IBM research publications; good license posture.	8B fits small GPUs; 20B/34B benefit from 24GB-48GB+ VRAM.
Meta Llama 3.1 8B/70B Instruct	Meta, United States	General-purpose instruction model family; useful for reasoning, documentation, and network-change planning.	8B fits C240 M5 easily; 70B needs 48GB+ quantized or multi-GPU.
BigCode StarCoder2 3B/7B/15B	BigCode / ServiceNow-Hugging Face collaboration	Code-focused open model trained on 600+ programming languages; useful as lightweight code model.	15B fits 24GB+ GPU; smaller versions can run on modest hardware.
Google Gemma 4 26B/31B and Gemma 3 12B/27B	Google DeepMind, United States / United Kingdom	Good non-Chinese general reasoning and agentic workflow option. Gemma 4 is especially attractive because Google describes it as Apache 2.0 licensed, with code generation, tool/function-calling, and long context.	Gemma 4 26B/31B fits best on 48GB-80GB+ GPUs or quantized on high-end cards; Gemma 3 12B/27B and smaller Gemma 4 E2B/E4B are better C240 M5 candidates.
Google CodeGemma 2B/7B	Google DeepMind, United States / United Kingdom	Lightweight code-specialized option for code completion, code generation, and code chat. Useful when the C240 M5 GPU budget is limited.	2B/7B fit T4/L4-class GPUs well; 7B instruction-tuned variant is appropriate for chat-style coding assistance.

Model family	Origin / steward	Why it fits	Hardware fit
Nomic Embed / IBM Granite Embeddings	US/Western alternatives	Local embeddings for retrieval over Cisco/Aruba docs and code repositories.	CPU or small GPU friendly.

Codestral is described by Mistral as an open-weight model explicitly designed for code-generation tasks. IBM's Granite Code work describes a model family for code intelligence across 3B to 34B parameters and states that the models were released under Apache 2.0. StarCoder2-15B is described as trained on 600+ programming languages from The Stack v2. Meta's Llama 3.1 model card records a July 23, 2024 release date. Google's Gemma 4 announcement states that Gemma 4 is released under Apache 2.0, includes code generation, native function-calling/structured output, and long context up to 256K on larger models. Google's CodeGemma documentation identifies 2B and 7B code-specialized variants for code completion, generation, and instruction-following. [4][5][6][7][17][18][19]

3.1 Assessment: Should Google Gemma Be Included?

Yes. Google Gemma should be included as an approved design option, with two roles. First, Gemma 4 should be considered a strong non-Chinese general/agent model option because Google describes it as Apache 2.0 licensed, built for reasoning and agentic workflows, and capable of offline code generation with long context. Second, CodeGemma should be included as a lightweight code-specialized option for constrained C240 M5 builds, especially where T4 or L4-class GPUs limit VRAM.

Recommended placement: use Gemma 4 26B/31B on modern double-width GPU servers with 48GB-80GB+ VRAM per GPU, use Gemma 3 12B/27B or smaller Gemma 4 E2B/E4B where hardware is constrained, and use CodeGemma 7B-IT or CodeGemma 2B for lightweight coding/chat/completion tasks on a C240 M5. For production use, keep Codestral, Granite Code, Llama, StarCoder2, and Gemma in the evaluation matrix and select based on accuracy against your internal coding, Cisco, and Aruba SD-WAN test set.

Caution: Gemma 4 has the cleanest license posture in the Gemma family because Google now describes it as Apache 2.0. Older Gemma 3 and CodeGemma releases may use Google Gemma terms rather than Apache 2.0, so the organization should review the exact model card and terms before moving them into the air-gapped lab.

4. Server and GPU Design Options

4.1 Cisco UCS C240 M5 Candidate Designs

The C240 M5 is viable for a constrained air-gapped lab, but it is not the best modern AI server. It is strongest when the goal is reuse of existing hardware, moderate-size models, and command-line coding assistance rather than heavy multi-user inference.

C240 M5 option	GPU approach	Expected model class	Pros	Cautions
M5-A: Low-risk reuse build	1-6 NVIDIA T4 16GB single-width cards, or similar validated single-width supported cards	Granite 8B, StarCoder2 7B/15B quantized, Llama 3.1 8B, CodeGemma 2B/7B, Gemma 4 E2B/E4B, small embeddings	Cheapest if you already own the server; low power; easier thermals.	Limited VRAM per card; not ideal for 70B-class models; PCIe Gen3.
M5-B: Better inference build	1-2 NVIDIA L4 24GB single-width cards if mechanically/power	Codestral 22B quantized, Granite 20B/34B quantized,	More useful VRAM than T4; 72W low-power profile; good	L4 may not be an official C240 M5 option; validate

C240 M5 option	GPU approach	Expected model class	Pros	Cautions
	validated in your exact chassis, riser, BIOS/CIMC, and OS stack	Gemma 3 12B/27B quantized, Llama 3.1 8B, StarCoder2 15B, CodeGemma 7B	for offline inference.	support before purchase.
M5-C: Legacy double-width build	Up to two supported double-width GPUs in approved risers; examples from Cisco sources include V100/A100-era and Quadro RTX 6000/8000-era configurations	Codestral 22B, Granite 34B, Gemma 4 26B/31B quantized, Llama 3.1 70B quantized if VRAM is sufficient	Uses true double-width GPUs; can make M5 more capable.	High thermal/power/riser sensitivity; do not assume modern L40S/H100 compatibility.

Cisco documentation for the C240 M5 states that up to two double-wide GPU cards are supported in particular riser slots, and that double-wide cards are not supported in all riser options. Cisco's C240 M5 SFF spec sheet also lists NVIDIA T4 16GB single-width support up to six cards, M10 dual-width support up to two cards, and Tesla A100 80GB dual-width support up to two cards in the cited spec table. Cisco also published a virtualization/GPU design note stating support for up to two NVIDIA RTX 6000 or RTX 8000 cards per C240 M5 in that context. [8][9][10]

Practical recommendation for C240 M5: use it as a proof-of-concept or single-user lab server. If you already own one, start with a validated T4/L4-class single-width design or a known-supported double-width configuration. Do not buy expensive modern GPUs for the M5 until Cisco compatibility, risers, power cables, fan kits, firmware, and thermals are validated.

4.2 Servers That Support Modern Double-Width GPUs

Server	Double-width GPU support	Best use	Notes
Cisco UCS C240 M7	Cisco installation guidance says C240 M7 can support up to three NVIDIA GPUs with Cisco IMC 4.3(1)+; the SFF spec sheet shows slots supporting single/double-wide GPU configurations.	Cisco-standard environment; moderate GPU density.	Good successor if you want to stay UCS and need newer PCIe Gen5 platform support.
Dell PowerEdge R760xa	Up to four double-width PCIe Gen5 accelerators or up to 12 single-width PCIe accelerators.	Balanced enterprise AI inference server.	Strong candidate for 2-4 L40S/RTX PRO/H100 PCIe-style deployments.
HPE ProLiant DL380a Gen11	HPE states support for up to four double-wide or eight single-wide GPUs in 2U.	Enterprise 2U GPU inference with HPE ecosystem.	Good general-purpose air-cooled GPU server.
Lenovo ThinkSystem SR675 V3	Supports up to eight double-wide full-height full-length 600W GPUs in dense module, or HGX H200 4-GPU option.	High-density inference/training and larger local LLMs.	Best fit when you want 4-8 GPUs or future growth.
Supermicro SYS-221GE-NR	2U GPU system with up to four NVIDIA PCIe GPUs;	Cost-effective high-density GPU server.	Often flexible and price-competitive; check enterprise support requirements.

Server	Double-width GPU support	Best use	Notes
	data sheet lists H100, H100 NVL, or L40S.		

For this project, the Dell R760xa and HPE DL380a Gen11 are practical enterprise options for up to four double-width GPUs. Lenovo SR675 V3 is the high-density option when more than four GPUs or HGX configurations are desired. Supermicro SYS-221GE-NR is attractive when hardware flexibility and price matter. Cisco UCS C240 M7 is the cleanest Cisco-aligned successor to an M5, although its exact GPU count depends on riser and firmware configuration. [11][12][13][14][15]

5. Recommended Bills of Materials

5.1 BOM A: Reuse Cisco UCS C240 M5

Component	Recommendation	Estimated price
Existing server	Cisco UCS C240 M5, dual Xeon Scalable CPUs, latest supported CIMC/BIOS	Already owned / \$800-\$2,500 used market
Memory	384GB-768GB DDR4 ECC if available; 256GB minimum	\$300-\$1,500 depending current installed RAM
GPU option 1	2-6 NVIDIA T4 16GB single-width GPUs	\$500-\$1,200 each used/refurbished
GPU option 2	1-2 NVIDIA L4 24GB single-width GPUs if validated	\$2,000-\$3,000 each based on 2026 market estimates
Storage	2 x 3.84TB or 7.68TB enterprise SSD for OS/model/docs	\$500-\$1,800 total
Networking	Lab-only management plus data VLAN; no internet default route	\$0-\$500
UPS	1500VA-2200VA UPS	\$300-\$900
Transfer media	Two encrypted 4TB-8TB external SSDs	\$500-\$1,400

Estimated total if server is already owned: \$3,000-\$10,000 depending GPU choice and storage. This is the economical path but not the highest-quality AI experience.

5.2 BOM B: Recommended Enterprise Double-Width Server

Component	Recommendation	Estimated price
Server chassis	Dell PowerEdge R760xa, HPE DL380a Gen11, or Cisco UCS C240 M7 configured for double-width GPUs	\$8,000-\$20,000 before GPUs
CPU	Dual Intel Xeon Scalable, mid-core-count SKUs	\$2,000-\$8,000 included/config dependent
Memory	512GB DDR5 ECC minimum; 1TB preferred for multi-user and large context	\$2,000-\$8,000
GPU baseline	2 x NVIDIA L40S 48GB or RTX 6000 Ada 48GB-class GPUs	\$6,500-\$9,000 each for RTX 6000 Ada-class retail examples; L40S often quote-based

Component	Recommendation	Estimated price
GPU growth	4 x L40S / RTX PRO / H100 PCIe-style GPUs if supported	\$26,000-\$120,000+ depending GPU family
Storage	OS: 2 x 1.92TB SSD mirror; model/docs: 4 x 7.68TB NVMe/SAS SSD	\$3,000-\$12,000
Network	10/25GbE lab connectivity; isolated management	\$1,000-\$4,000
UPS	2200VA-3000VA UPS	\$800-\$2,500
Transfer media	Two encrypted 8TB SSDs	\$800-\$1,600

Estimated total for a strong 2-GPU system: \$28,000-\$55,000. Estimated total for a 4-GPU system: \$55,000-\$150,000+, depending on GPU family and OEM support contract.

5.3 BOM C: High-Density AI Server

Component	Recommendation	Estimated price
Server chassis	Lenovo SR675 V3 or Supermicro multi-GPU system	\$15,000-\$45,000 before GPUs
CPU/memory	Dual EPYC or Xeon platform, 1TB-2TB RAM	\$8,000-\$25,000
GPU	4-8 x L40S, H100 NVL, H200, RTX PRO 6000 Server Edition, or equivalent	\$60,000-\$300,000+
Storage	30TB-60TB NVMe/U.2/U.3 usable	\$8,000-\$30,000
Power/cooling	Rack power and cooling validation required	Site dependent
Estimated total	High-density local LLM server	\$100,000-\$400,000+

6. Software BOM

Layer	Software	Reason
OS	Ubuntu Server 24.04 LTS or Rocky Linux 9	Stable Linux platform with good NVIDIA and container support.
GPU stack	NVIDIA driver, CUDA runtime, NVIDIA Container Toolkit	Required for GPU inference in containers.
Model runtime	vLLM	Primary local model API; OpenAI-compatible server.
Simple runtime	Ollama	Simple local model runner and CLI fallback.
Quantized fallback	llama.cpp	GGUF model testing and CPU/small-GPU fallback.
Coding CLI	Aider	Terminal pair-programming against Git repositories.
Coding CLI	OpenCode	Provider-agnostic terminal coding agent; validate exact current fork/version during staging.
Retrieval	Qdrant, Chroma, or LanceDB	Local vector database for Cisco/Aruba docs and internal standards.
Embeddings	Nomic Embed or IBM Granite Embeddings	Non-Chinese local embedding models for search.
Package control	Python wheelhouse, npm/pnpm cache, local apt/yum mirror	Offline install and reproducibility.
Security	nftables/iptables, auditd, local logs	Air-gap verification and change tracking.

7. Internet-Connected Staging Build Steps

7.1 Prepare directories

```
mkdir -p ~/airgap-ai-build/{repos,models,containers,wheels,npm,docs,manifests,scripts}
cd ~/airgap-ai-build
```

7.2 Clone repositories

```
cd ~/airgap-ai-build/repos
git clone https://github.com/vllm-project/vllm.git
git clone https://github.com/ollama/ollama.git
git clone https://github.com/ggml-org/llama.cpp.git
git clone https://github.com/aider-ai/aider.git
git clone https://github.com/sst/opencode.git || true
# If the OpenCode project location changes, pin the validated repo URL and commit in
manifests/build-notes.txt.
```

7.3 Download approved model weights

```
python3 -m venv ~/airgap-ai-build/venv
source ~/airgap-ai-build/venv/bin/activate
pip install -U pip huggingface_hub hf_transfer
cd ~/airgap-ai-build/models

# Mistral Codestral
huggingface-cli download mistralai/Codestral-22B-v0.1 --local-dir Codestral-22B-v0.1 --local-dir-use-symlinks False

# IBM Granite Code examples. Adjust exact model names after license approval.
huggingface-cli download ibm-granite/granite-20b-code-instruct --local-dir granite-20b-code-instruct --local-dir-use-symlinks False || true
huggingface-cli download ibm-granite/granite-34b-code-instruct --local-dir granite-34b-code-instruct --local-dir-use-symlinks False || true

# Meta Llama requires license acceptance on Hugging Face or Meta.
huggingface-cli download meta-llama/Llama-3.1-8B-Instruct --local-dir Llama-3.1-8B-Instruct --local-dir-use-symlinks False
huggingface-cli download meta-llama/Llama-3.1-70B-Instruct --local-dir Llama-3.1-70B-Instruct --local-dir-use-symlinks False

# BigCode StarCoder2
huggingface-cli download bigcode/starcoder2-15b --local-dir starcoder2-15b --local-dir-use-symlinks False

# Google Gemma / CodeGemma, subject to model-card and license approval
huggingface-cli download google/codegemma-7b-it --local-dir codegemma-7b-it --local-dir-use-symlinks False
huggingface-cli download google/gemma-4-31b-it --local-dir gemma-4-31b-it --local-dir-use-symlinks False
```

7.4 Pull and export containers

```
cd ~/airgap-ai-build
docker pull vllm/vllm-openai:latest
docker pull ollama/ollama:latest
docker pull qdrant/qdrant:latest

docker save vllm/vllm-openai:latest -o containers/vllm-openai_latest.tar
```

```
docker save ollama/ollama:latest -o containers/ollama_latest.tar
docker save qdrant/qdrant:latest -o containers/qdrant_latest.tar
```

7.5 Build a Python wheelhouse

```
cd ~/airgap-ai-build
cat > requirements-airgap.txt <<'EOF'
vllm
aider-chat
huggingface_hub
transformers
accelerate
sentence-transformers
qdrant-client
chromadb
llama-index
langchain
openai
rich
typer
fastapi
uvicorn
tree-sitter
EOF
pip download -r requirements-airgap.txt -d wheels
```

7.6 Download documentation and internal knowledge

Download official Cisco IOS XE/NX-OS command references, configuration guides, release notes for exact lab versions, Aruba SD-WAN documentation, internal standards, golden configurations, change-control templates, rollback templates, IP/VLAN plans, and device inventory. Store them under /docs with platform/version metadata. Where licensing prevents redistribution, move only the content your organization is allowed to store offline.

7.7 Generate manifests

```
cd ~/airgap-ai-build
find . -type f -print0 | sort -z | xargs -0 sha256sum > manifests/SHA256SUMS.txt
cat > manifests/model-policy.txt <<'EOF'
Excluded: Qwen, DeepSeek, InternLM, Yi, Baichuan, MiniCPM, GLM,
Chinese-origin models, and Chinese-distilled derivatives.
Approved candidates: Mistral Codestral, IBM Granite Code, Meta Llama, BigCode StarCoder2.
EOF
```

8. Air-Gap Import and Offline Installation

```
# On air-gapped host after removable-media approval
sudo mkdir -p /opt/airgap-ai
sudo rsync -a /media/transfer/airgap-ai-build/ /opt/airgap-ai/
cd /opt/airgap-ai
sha256sum -c manifests/SHA256SUMS.txt

# Install Python packages offline
python3 -m venv /opt/airgap-ai/venv
source /opt/airgap-ai/venv/bin/activate
pip install --no-index --find-links=/opt/airgap-ai/wheels aider-chat transformers accelerate qdrant-client

# Load containers
for i in /opt/airgap-ai/containers/*.tar; do docker load -i "$i"; done
```

8.1 Start vLLM locally

```
docker run --gpus all --ipc=host --rm -p 8000:8000 -v /opt/airgap-ai/models:/models vllm/vllm-openai:latest --model /models/Codestral-22B-v0.1 --served-model-name local-coder --host 0.0.0.0 --port 8000 --max-model-len 32768
```

8.2 Test the local endpoint

```
curl http://127.0.0.1:8000/v1/models
curl http://127.0.0.1:8000/v1/chat/completions -H "Content-Type: application/json" -d '{"model": "local-coder", "messages": [{"role": "user", "content": "Write a Cisco interface validation function in Python."}], "temperature": 0.2}'
```

8.3 Use Aider with the local model

```
export OPENAI_API_BASE=http://127.0.0.1:8000/v1
export OPENAI_API_KEY=local-not-used
cd /path/to/git/repo
aider --model openai/local-coder
```

9. Cisco and Aruba Retrieval-Augmented Design

For network engineering, the model should not be allowed to invent vendor commands from memory. The assistant should retrieve from local Cisco and Aruba documentation, internal standards, golden configs, and previous approved change records before proposing configurations.

RAG collection	Examples	Metadata
Cisco docs	IOS XE, NX-OS, BGP, OSPF, VLANs, ACLs, QoS, SNMP, AAA, logging	vendor, platform, version, topic, source, date
Aruba docs	SD-WAN appliance install/config/troubleshooting guides	vendor, product, version, feature, source, date
Internal standards	IP plan, VLAN naming, routing policy, NTP/SNMP/TACACS standards	owner, approval date, classification
Golden configs	Baseline router/switch/SD-WAN configs	device type, site type, version, approved
Change records	Prechecks, change steps, validations, rollback	change ID, risk, outcome, approver

The network-change response format should always include assumptions, device/platform/version, current-state data required, risk assessment, prechecks, proposed configuration, implementation sequence, validation commands, rollback commands, outage impact, and human approval checklist.

10. Timeline

Phase	Work	Estimated duration
0. Requirements	Confirm non-Chinese model policy, lab device versions, users, security boundary, model-size target	0.5-1 day
1. Hardware decision	Choose C240 M5 reuse vs new double-width GPU server; verify GPU compatibility and power/cooling	2-10 days
2. Connected staging	Install Linux, GPU drivers, Docker/Podman, Python, Node; validate GPU runtime	0.5-1.5 days
3. Tool validation	Test vLLM, Ollama, llama.cpp, Aider, OpenCode against sample repos	1-2 days
4. Model evaluation	Evaluate Codestral, Granite Code, Llama, StarCoder2 on code and network tasks	2-4 days

Phase	Work	Estimated duration
5. Documentation library	Download and organize Cisco/Aruba/internal docs	1-3 days
6. Retrieval/indexing	Chunk docs, build local embeddings, test retrieval quality	1-3 days
7. Air-gap bundle	Export containers, wheels, npm cache, docs, models, checksums, SBOM	0.5-1 day
8. Transfer/install	Move through media control, verify hashes, install offline	0.5-1 day
9. Acceptance testing	Coding tests, Cisco config tests, Aruba SD-WAN tests, no-egress validation	2-5 days
10. Runbook/training	Document operations, model update process, approval workflow	1-2 days

Expected active effort: about 10-24 working days excluding procurement lead time.

11. Recommendation

Best reuse path: If the UCS C240 M5 is already available, use it for a limited single-user or proof-of-concept build with T4/L4-class single-width GPUs or a known-supported legacy double-width GPU configuration. Run Codestral 22B, Granite Code 20B/34B, StarCoder2 15B, and Llama 3.1 8B in quantized or memory-appropriate forms. This path is cost-effective but limited by PCIe Gen3, VRAM, thermals, and compatibility.

Best production-like path: Buy a modern double-width GPU server such as Dell PowerEdge R760xa or HPE ProLiant DL380a Gen11 with two to four 48GB-class GPUs. This gives a much better experience for medium-size codebases, longer context windows, and local network-documentation retrieval. If the environment is Cisco-standardized, evaluate UCS C240 M7. If high density is required, evaluate Lenovo SR675 V3 or Supermicro SYS-221GE-NR.

Best model plan: Use Codestral 22B or IBM Granite Code as the primary code model, Llama 3.1 70B as a stronger general reasoning/planning model when hardware allows, StarCoder2 as a lightweight coding fallback, and Nomic/Granite embeddings for local retrieval. Exclude all Chinese-origin and Chinese-distilled model families in the manifest and acceptance checklist.

12. References

- [1] vLLM OpenAI-compatible server documentation, https://docs.vllm.ai/en/latest/serving/openai_compatible_server/
- [2] Aider GitHub repository and documentation, <https://github.com/aider-ai/aider> and <https://aider.chat/docs/>
- [3] OpenCode repository, <https://github.com/sst/opencode> and provider-agnostic coding-agent information, <https://github.com/anomalyco/opencode>
- [4] Mistral AI, Codestral announcement, <https://mistral.ai/news/codestral>; Hugging Face model card <https://huggingface.co/mistralai/Codestral-22B-v0.1>
- [5] IBM Granite Code Models technical paper, <https://arxiv.org/abs/2405.04324>; long-context Granite Code paper, <https://arxiv.org/abs/2407.13739>
- [6] Meta Llama 3.1 model card, <https://huggingface.co/meta-llama/Llama-3.1-70B>; Meta Llama 3.1 announcement, <https://ai.meta.com/blog/meta-llama-3-1/>
- [7] BigCode StarCoder2 model card, <https://huggingface.co/bigcode/starcoder2-15b>; StarCoder2 repository, <https://github.com/bigcode-project/starcoder2>

- [8] Cisco UCS C240 M5 GPU installation guidance, https://www.cisco.com/c/en/us/td/docs/unified_computing/ucs/c/hw/C240M5/install/C240M5/C240M5_appendix_0101.html
- [9] Cisco UCS C240 M5 SFF rack server spec sheet, <https://www.cisco.com/c/dam/en/us/products/collateral/servers-unified-computing/ucs-c-series-rack-servers/c240m5-sff-specsheet.pdf>
- [10] Cisco UCS M5 with NVIDIA GRID/Citrix design note, <https://www.cisco.com/c/en/us/products/collateral/servers-unified-computing/ucs-c-series-rack-servers/ucs-m5-with-nvidia-and-citrix.html>
- [11] Cisco UCS C240 M7 GPU installation guidance and SFF spec sheet, https://www.cisco.com/c/en/us/td/docs/unified_computing/ucs/c/hw/C240M7/install/b-cisco-c240-m7-install/m-gpu-install.html and <https://www.cisco.com/c/dam/en/us/products/collateral/servers-unified-computing/ucs-c-series-rack-servers/c240m7-sff-specsheet.pdf>
- [12] Dell PowerEdge R760xa spec sheet, <https://www.delltechnologies.com/asset/no-no/products/servers/technical-support/poweredge-r760xa-spec-sheet.pdf>
- [13] HPE ProLiant DL380a Gen11 product and QuickSpecs, <https://www.hpe.com/us/en/compute/hpe-proliant-compute/dl380a-gen11.html> and <https://www.hpe.com/us/en/collaterals/collateral.a50004309enw.html>
- [14] Lenovo ThinkSystem SR675 V3 data sheet, <https://lenovopress.lenovo.com/datasheet/ds0151-lenovo-thinksystem-sr675-v3>
- [15] Supermicro SYS-221GE-NR data sheet, <https://www.supermicro.com/en/products/system/datasheet/sys-221ge-nr>
- [16] NVIDIA L4 pricing estimate, JarvisLabs, <https://jarvislabs.ai/blog/l4-gpu-price>; RTX 6000 Ada retail reference, <https://networkoutlet.com/products/nvidia-rtx-6000-ada-48gb-gddr6-professional-gpu-ai-rendering-amp-simulation>
- [17] Google Gemma 4 announcement: Apache 2.0 license, code generation, agentic workflows, and long context. <https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/>
- [18] Google Gemma release notes: Gemma 4 sizes and release dates. <https://ai.google.dev/gemma/docs/releases>
- [19] Google CodeGemma documentation and model card: 2B/7B code completion, generation, and instruction-tuned variants. <https://ai.google.dev/gemma/docs/codegemma> and <https://huggingface.co/google/codegemma-7b-it>
- [20] Google Gemma 3 model card: 1B/4B/12B/27B sizes and 128K context for 4B/12B/27B. https://ai.google.dev/gemma/docs/core/model_card_3